Project Report

Loan Approved or Not?

Decision Tree Classification

Name	Samar Rathod
Roll No.	B-42
Enrollment No.	24004501210359
Division	D
Project ID	P36
Project Title	Loan Approved or Not? (Decision Tree Classification)

1. Problem Statement

A bank must decide whether each loan application should be approved. Doing this by hand is slow and can be inconsistent. This project builds a **Decision Tree classifier** that predicts **Loan_Status** (Y = approved, N = not approved) from applicant information: gender, marital status, dependents, education, employment type, income, co-applicant income, loan amount, loan term, credit history and property area.

This is a **binary classification** problem. The tree is also easy to read, so it shows *which factors drive an approval*, not just the prediction.

2. Dataset Description

The dataset is **train.csv** (the Loan Prediction dataset): **614 applications**, **13 columns**, no duplicate rows. The target is imbalanced: **422 approved (68.7%)** and **192 not approved (31.3%)**. Several columns have missing values (up to 50 in Credit_History).

Data dictionary

Column	Type	Meaning	Missing	Unique
Loan_ID	str	Unique application ID (dropped; not a feature)	0	614
Gender	str	Male / Female	13	2
Married	str	Applicant married (Yes / No)	3	2
Dependents	str	Number of dependents (0, 1, 2, 3+)	15	4
Education	str	Graduate / Not Graduate	0	2
Self_Employed	str	Self-employed (Yes / No)	32	2
ApplicantIncome	int64	Applicant monthly income	0	505
CoapplicantIncome	float64	Co-applicant monthly income	0	287
LoanAmount	float64	Loan amount (in thousands)	22	203
Loan_Amount_Term	float64	Loan term in months	14	10
Credit_History	float64	1 = meets credit guidelines, 0 = does not	50	2
Property_Area	str	Urban / Semiurban / Rural	0	3
Loan_Status	str	TARGET: Y = approved, N = not approved	0	2

Descriptive statistics: median ApplicantIncome is 3,812 (mean 5,403, max 81,000), so income is strongly right-skewed. Median LoanAmount is 128 and the most common loan term is 360 months. About 84% of applicants with a recorded credit history have Credit_History = 1.

3. Data Cleaning and EDA

Cleaning steps

- Dropped duplicate rows (none found) and removed **Loan_ID**, because an ID says nothing about eligibility.
- Created a new feature **TotalIncome = ApplicantIncome + CoapplicantIncome**.
- Standardised the target labels (strip spaces, upper-case) and stripped spaces in text columns. All 614 rows had a valid target.
- One-hot encoded the categorical columns with `get_dummies(drop_first=True)`, giving 15 numeric features.
- Missing values were filled with the **median** using `SimpleImputer`, fitted on the **training set only** to avoid data leakage.

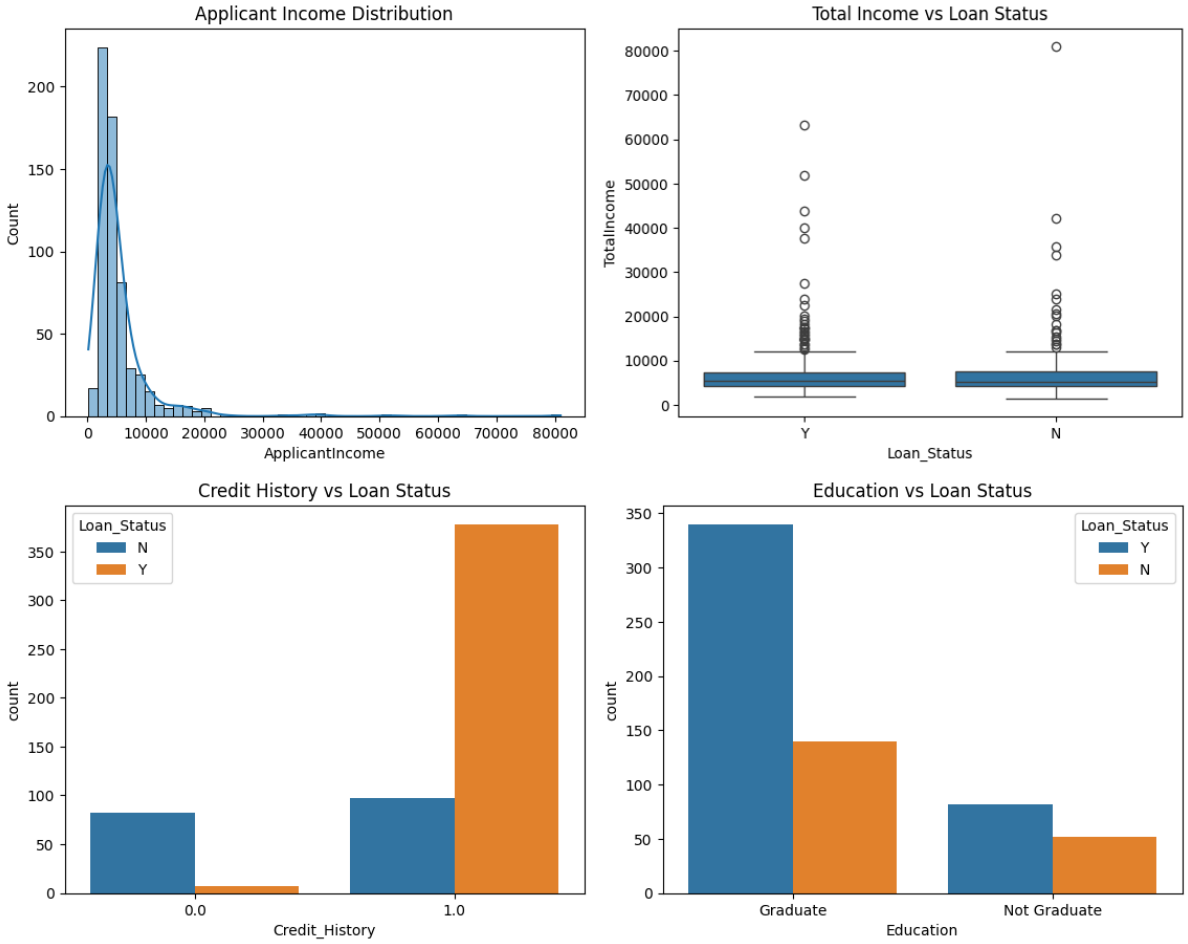


Fig 1. Applicant income distribution, total income by loan status, credit history vs loan status, education vs loan status.

Key insights

- **Credit history is the strongest signal.** Roughly 80% of applicants with Credit_History = 1 were approved, versus under 10% of those with 0.
- **Income alone does not separate the classes.** The TotalIncome boxplots for approved and rejected look almost identical; income is skewed with many high outliers.
- **Education has a small effect.** Graduates were approved about 71% of the time and non-graduates about 61%.
- **Class imbalance:** 68.7% approved, so a model that always says "approved" already scores about 69% accuracy. This is the baseline to beat.

4. Models and Settings Tried

Setup

- **Split:** 80% train (491) / 20% test (123), `random_state=42`, stratified on the target so both sets keep the same approved/rejected ratio.
- **Feature selection:** `SelectKBest(f_classif, k=10)` on the training data. Kept: CoapplicantIncome, Loan_Amount_Term, Credit_History, TotalIncome, Married_Yes, Dependents_1, Dependents_2, Education_Not Graduate, Property_Area_Semiurban, Property_Area_Urban.
- **Baseline model:** `DecisionTreeClassifier(random_state=42)` - gini, unlimited depth.
- **Experiment:** 2 split criteria (**gini**, **entropy**) x 6 values of **max_depth** (2, 3, 5, 7, 9, None) = 12 trees, compared on accuracy, precision, recall and F1. The best F1 was chosen as the final model.

All 12 settings (sorted by F1)

Criterion	Max depth	Accuracy	Precision	Recall	F1
entropy	**5**	**0.854**	**0.838**	**0.976**	**0.902**
entropy	2	0.854	0.845	0.965	0.901
entropy	3	0.854	0.853	0.953	0.900
gini	5	0.846	0.830	0.976	0.897
gini	2	0.846	0.837	0.965	0.896
gini	3	0.846	0.844	0.953	0.895
gini	7	0.829	0.814	0.976	0.888
entropy	7	0.829	0.814	0.976	0.888
entropy	9	0.829	0.820	0.965	0.886
gini	9	0.805	0.821	0.918	0.867
gini	None (baseline)	0.772	0.813	0.871	0.841
entropy	None	0.683	0.788	0.741	0.764

Green row = selected final model (entropy, max_depth = 5).

What the table shows

- **Unlimited depth overfits.** The fully grown trees have the lowest scores (entropy 0.683, gini 0.772 accuracy). Limiting depth to 2-5 raised accuracy by 8-17 points.
- **Shallow trees are about as good as depth 5.** Depths 2, 3 and 5 are within 0.002 F1 of each other, so a simpler tree (depth 2 or 3) would be nearly as good and easier to explain.
- Entropy and gini behave almost the same at shallow depth; the difference is small.

5. Results

Baseline vs final model (test set, 123 applications)

Model	Accuracy	Precision	Recall	F1	AUC
Majority class ("always approve")	0.691	-	-	-	0.500
Baseline tree (gini, depth None)	0.772	0.813	0.871	0.841	-
Final tree (entropy, depth 5)	**0.854**	**0.838**	**0.976**	**0.902**	**0.699**

Precision, recall and F1 are for the "Approved" class. AUC for the baseline was not computed in the notebook; the majority-class row is a reference computed from the test labels (85 of 123 approved).

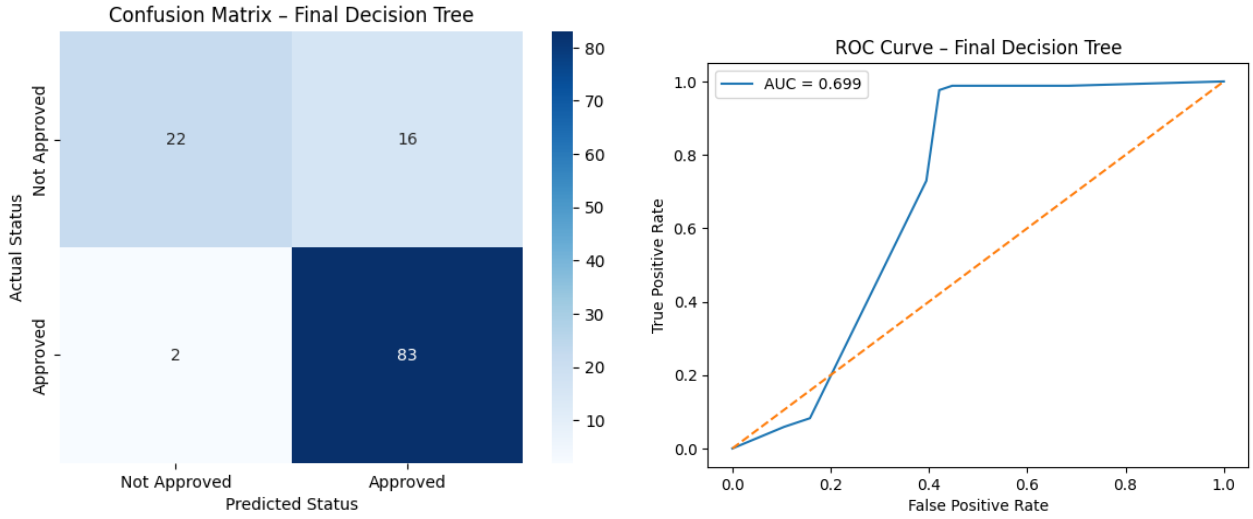


Fig 2. Confusion matrix (left) and ROC curve with AUC = 0.699 (right) for the final model.

Reading the confusion matrix

- **83** approved applications correctly predicted approved (TP), **22** rejected correctly predicted rejected (TN).
- **16** rejected applications were wrongly predicted as approved (FP) and only **2** approved ones were missed (FN).
- Recall for Approved is 83/85 = 97.6%, but recall for Not Approved is only 22/38 = 57.9%. The model is very good at finding approvals but lets many risky applications through. For a bank, those false approvals are the costly mistakes.

Feature importance and sample predictions

Credit_History (0.646) and **TotalIncome (0.263)** together account for about 91% of the tree's decisions. Everything else is under 0.03. Three test applications were scored: #1 Not Approved (0.00% approval probability), #2 Approved (82.48%), #3 Approved (70.37%).

6. Conclusion and What I Learned

Conclusion

A Decision Tree with **entropy** and **max_depth = 5** predicted loan approval with **85.4% accuracy** and an **F1 of 0.902** on unseen data, compared with 77.2% for the unrestricted baseline tree and about 69% for always guessing "approved". Credit history is by far the most important factor, followed by total income.

What I learned

- **Cleaning matters:** handling missing values, dropping an ID column and encoding categories had to be done before any model could run.
- **Avoid data leakage:** the imputer and the feature selector were fitted on training data only, then applied to the test data.
- **Overfitting is real:** the fully grown tree did worst. Limiting max_depth was the single biggest improvement.
- **Accuracy is not enough:** with 69% approvals, accuracy looks good even for a weak model. The confusion matrix and ROC curve showed the weakness on rejected loans.
- Feature engineering (TotalIncome) was useful: it became the second most important feature.

Limitations and next steps

- The final settings were picked by comparing scores on the **test set**, so 85.4% is slightly optimistic. A better method is to tune with cross-validation (e.g. GridSearchCV) and keep the test set untouched.
- The AUC of 0.699 is modest because a single tree outputs only a few distinct probabilities. Try Random Forest or Gradient Boosting, and use `class_weight="balanced"` to improve recall on rejected loans.
- The test set is small (123 rows), so results can change with a different split. SelectKBest dropped ApplicantIncome and LoanAmount; it is worth testing the model with them kept.
- The dataset is small and historical. The model should support, not replace, a human loan officer.